

Отчёт по лабораторной работе №2

Компьютерный практикум по статистическому анализу

Надежда Александровна Рогожина

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
4	Выполнение лабораторной работы	8
5	Выводы	33
	Список литературы	34

Список иллюстраций

4.1	Кортежи	8
4.2	Словари	9
4.3	Словари	9
4.4	Множества	10
4.5	Множества	11
4.6	Массивы	12
4.7	Массивы	13
4.8	Массивы	14
4.9	Массивы	15
4.10	Массивы	16
4.11	Массивы	17
4.12	Массивы	18
4.13	Массивы	19
4.14	Объединение множеств	20
4.15	Собственные примеры	20
4.16	Работа с массивами	23
4.17	Работа с массивами	24
4.18	Работа с массивами	24
4.19	Работа с массивами	25
4.20	Работа с массивами	26
4.21	Работа с массивами	27
4.22	Работа с массивами	28
4.23	Работа с массивами	29
4.24	Работа с массивами	30
4.25	Работа с массивами	31
4.26	Работа с массивами	32

Список таблиц

1 Цель работы

Основная цель работы — изучить несколько структур данных, реализованных в Julia, научиться применять их и операции над ними для решения задач.

2 Задание

1. Используя Jupyter Lab, повторите примеры из раздела 2.2.
2. Выполните задания для самостоятельной работы (раздел 2.4).

3 Теоретическое введение

Научные вычисления традиционно требуют высочайшей производительности, однако эксперты по доменам в значительной степени перенесены на более медленные динамические языки для ежедневной работы. К счастью, методы современного языка и компилятора позволяют в основном устранить компромисс производительности и обеспечивать достаточно продуктивную среду для прототипирования и достаточно эффективно для развертывания применений, интенсивных производительности. Язык программирования Julia заполняет эту роль: это гибкий динамический язык, подходящий для научных и численных вычислений, с производительностью, сравнимой с традиционными статичными языками.

4 Выполнение лабораторной работы

Первым заданием было повторить примеры, где были показаны основы работы с кортежами (рис. 4.1), словарями (рис. 4.2, рис. 4.3), множествами (рис. 4.4, рис. 4.5) и массивами (рис. 4.6, рис. 4.7, рис. 4.8, рис. 4.9, рис. 4.10, рис. 4.11, рис. 4.12, рис. 4.13).

```
Рассмотрим несколько структур данных, реализованных в Julia. Несколько функций (методов), общих для всех структур данных:  
• isempty() — проверяет, пуста ли структура данных;  
• length() — возвращает длину структуры данных;  
• in() — проверяет принадлежность элемента к структуре;  
• unique() — возвращает коллекцию уникальных элементов структуры;  
• reduce() — сворачивает структуру данных в соответствии с заданным бинарным оператором;  
• maximum() (или minimum()) — возвращает наибольший (или наименьший) результат вложенной функции для каждого элемента структуры данных.  
  
In [1]: ()  
Out[1]: ()  
  
In [5]: FavoriteLang = ("Python", "Julia", "R")  
Out[5]: ("Python", "Julia", "R")  
  
In [3]: x2 = (1, 2, 3)  
Out[3]: (1, 2, 3)  
  
In [4]: x2 = (1, 2.0, "one")  
Out[4]: (1, 2.0, "one")  
  
In [6]: x3 = (a=2, b=1+2)  
Out[6]: {a = 2, b = 3}  
  
In [8]: length(x3)  
Out[8]: 2  
  
In [10]: x2[1], x2[2], x2[3]  
Out[10]: (1, 2.0, "one")  
  
In [11]: c = x2[2] + x2[3]  
Out[11]: 5  
  
In [12]: x3.a, x3.b, x3[2]  
Out[12]: (2, 3, 3)  
  
In [13]: is{true}, x3, 0 in x2  
Out[13]: (true, false)
```

Рисунок 4.1: Кортежи


```

Словари

In [28]: phonebook = Dict("Манеж В.В." => ("907-5309", "333-5544"), "Бухаринев" => "555-2368")
Out[28]: Dict{String, Any} with 2 entries:
  "Бухаринев" => "555-2368"
  "Манеж В.В." => ("907-5309", "333-5544")

In [29]: keys(phonebook)
Out[29]: KeySet for a Dict{String, Any} with 2 entries. Keys:
  "Бухаринев"
  "Манеж В.В."

In [30]: values(phonebook)
Out[30]: ValueIterator for a Dict{String, Any} with 2 entries. Values:
  "555-2368"
  ("907-5309", "333-5544")

In [31]: pairs(phonebook)
Out[31]: Dict{String, Any} with 2 entries:
  "Бухаринев" => "555-2368"
  "Манеж В.В." => ("907-5309", "333-5544")

In [32]: haskey(phonebook, "Манеж В.В.")
Out[32]: true

In [33]: phonebook["Каппон Н.С."] = "555-3344"
Out[33]: Dict{String, Any} with 3 entries:
  "Каппон Н.С." => "555-3344"
  "Бухаринев" => "555-2368"
  "Манеж В.В." => ("907-5309", "333-5544")

In [34]: pop!(phonebook, "Манеж В.В.")
Out[34]: Dict{String, Any} with 2 entries:
  "Каппон Н.С." => "555-3344"
  "Бухаринев" => "555-2368"

```

Рисунок 4.2: Словари

```

In [35]: a = Dict{"foo" => 0.0, "bar" => 42.0};
          b = Dict{"baz" => 17, "bar" => 13.0};
          merge(a,b)

```

```

Out[35]: Dict{String, Real} with 3 entries:
  "bar" => 13.0
  "baz" => 17
  "foo" => 0.0

```

```

In [36]: a = Dict{"foo" => 0.0, "bar" => 42.0};
          b = Dict{"baz" => 17, "bar" => 13.0};
          merge(b,a)

```

```

Out[36]: Dict{String, Real} with 3 entries:
  "bar" => 42.0
  "baz" => 17
  "foo" => 0.0

```

Рисунок 4.3: Словари

Множества

```
In [49]: A = Set([1, 3, 4, 5])
```

```
Out[49]: Set{Int64} with 4 elements:  
5  
4  
3  
1
```

```
In [50]: B = Set("abracadabra")
```

```
Out[50]: Set{Char} with 5 elements:  
'a'  
'd'  
'r'  
'k'  
'b'
```

```
In [51]: S1 = Set([1,2]);  
S2 = Set([3,4]);  
issetequal(S1,S2)
```

```
Out[51]: false
```

```
In [52]: S3 = Set([1,2,2,3,1,2,3,2,1]);  
S4 = Set([2,3,1]);  
issetequal(S3,S4)
```

```
Out[52]: true
```

Рисунок 4.4: Множества

```
In [53]: C=union(S1,S2)
Out[53]: Set{Int64} with 4 elements:
         4
         2
         3
         1
```

```
In [54]: D = intersect(S1,S3)
Out[54]: Set{Int64} with 2 elements:
         2
         1
```

```
In [58]: E = setdiff(S4,S2)
Out[58]: Set{Int64} with 2 elements:
         2
         1
```

```
In [55]: issubset(S1,S4)
Out[55]: true
```

```
In [59]: push!(S4, 99)
Out[59]: Set{Int64} with 4 elements:
         2
        99
         3
         1
```

```
In [60]: pop!(S4)
Out[60]: 2
```

```
In [61]: S4
Out[61]: Set{Int64} with 3 elements:
        99
         3
         1
```

Рисунок 4.5: Множества

Массивы

```
In [65]: empty_array_3 = (Float64)[]
Out[65]: Float64[]

In [66]: a = [1, 2, 3]
Out[66]: 3-element Vector{Int64}:
 1
 2
 3

In [67]: b = [1 2 3]
Out[67]: 1×3 Matrix{Int64}:
 1 2 3

In [69]: A = [[1, 2, 3] [4, 5, 6] [7, 8, 9]]
Out[69]: 3×3 Matrix{Int64}:
 1 4 7
 2 5 8
 3 6 9

In [70]: B = [[1 2 3]; [4 5 6]; [7 8 9]]
Out[70]: 3×3 Matrix{Int64}:
 1 2 3
 4 5 6
 7 8 9

In [71]: c = rand(1,8)
Out[71]: 1×8 Matrix{Float64}:
 0.363351  0.438101  0.976741  0.727742  ... 0.538385  0.84316  0.685373

In [73]: C = rand(2,3)
Out[73]: 2×3 Matrix{Float64}:
 0.543983  0.116562  0.267335
 0.80741  0.0731718  0.485584

In [74]: D = rand(4, 3, 2)
Out[74]: 4×3×2 Array{Float64, 3}:
[:, :, 1] =
 0.0268109  0.705842  0.575112
 0.405165  0.20176  0.824653
 0.299097  0.844161  0.244776
 0.674948  0.759963  0.96651

[:, :, 2] =
 0.371976  0.833723  0.112767
 0.302967  0.383296  0.0421015
 0.301494  0.324486  0.90377
 0.307533  0.163487  0.495973
```

Рисунок 4.6: Массивы

```
In [75]: roots = [sqrt(i) for i in 1:10]
```

```
Out[75]: 10-element Vector{Float64}:
```

```
 1.0
 1.4142135623730951
 1.7320508075688772
 2.0
 2.23606797749979
 2.449489742783178
 2.6457513110645907
 2.8284271247461903
 3.0
 3.1622776601683795
```

```
In [76]: ar_1 = [3*i^2 for i in 1:2:9]
```

```
Out[76]: 5-element Vector{Int64}:
```

```
 3
 27
 75
 147
 243
```

```
In [77]: ar_2=[i^2 for i=1:10 if (i^2%5!=0 && i^2%4!=0)]
```

```
Out[77]: 4-element Vector{Int64}:
```

```
 1
 9
 49
 81
```

Рисунок 4.7: Массивы

```

In [78]: ones(5)
Out[78]: 5-element Vector{Float64}:
          1.0
          1.0
          1.0
          1.0
          1.0

In [79]: ones(2,3)
Out[79]: 2×3 Matrix{Float64}:
          1.0  1.0  1.0
          1.0  1.0  1.0

In [80]: zeros(4)
Out[80]: 4-element Vector{Float64}:
          0.0
          0.0
          0.0
          0.0

In [86]: fill(3.5,(3,2))
Out[86]: 3×2 Matrix{Float64}:
          3.5  3.5
          3.5  3.5
          3.5  3.5

In [89]: repeat([1,2],3,3)
Out[89]: 6×3 Matrix{Int64}:
          1  1  1
          2  2  2
          1  1  1
          2  2  2
          1  1  1
          2  2  2

In [88]: repeat([1 2],3,3)
Out[88]: 3×6 Matrix{Int64}:
          1  2  1  2  1  2
          1  2  1  2  1  2
          1  2  1  2  1  2

```

Рисунок 4.8: Массивы

```
In [90]: a = collect(1:12)
         b = reshape(a,(2,6))

Out[90]: 2×6 Matrix{Int64}:
          1  3  5  7  9 11
          2  4  6  8 10 12

In [93]: b'
```

```
Out[93]: 6×2 adjoint(::Matrix{Int64}) with eltype Int64:
          1  2
          3  4
          5  6
          7  8
          9 10
         11 12

In [95]: c = transpose(b)
```

```
Out[95]: 6×2 transpose(::Matrix{Int64}) with eltype Int64:
          1  2
          3  4
          5  6
          7  8
          9 10
         11 12
```

Рисунок 4.9: Массивы

```
In [96]: ar = rand(10:20, 10, 5)
```

```
Out[96]: 10×5 Matrix{Int64}:  
 15 19 18 13 16  
 10 11 18 10 15  
 19 12 19 18 15  
 15 12 16 13 16  
 13 17 18 14 20  
 10 20 20 18 10  
 18 10 15 12 10  
 13 20 17 16 13  
 17 16 19 16 11  
 11 11 10 14 16
```

```
In [100]: ar[:, 2]
```

```
Out[100]: 10-element Vector{Int64}:  
 19  
 11  
 12  
 12  
 17  
 20  
 10  
 20  
 16  
 11
```

```
In [98]: ar[:, [2, 5]]
```

```
Out[98]: 10×2 Matrix{Int64}:  
 19 16  
 11 15  
 12 15  
 12 16  
 17 20  
 20 10  
 10 10  
 20 13  
 16 11  
 11 16
```

Рисунок 4.10: Массивы


```
In [101]: ar[:, 2:4]
```

```
Out[101]: 10×3 Matrix{Int64}:  
 19 18 13  
 11 18 10  
 12 19 18  
 12 16 13  
 17 18 14  
 20 20 18  
 10 15 12  
 20 17 16  
 16 19 16  
 11 10 14
```

```
In [102]: ar[[2, 4, 6], [1, 5]]
```

```
Out[102]: 3×2 Matrix{Int64}:  
 10 15  
 15 16  
 10 10
```

```
In [103]: ar[1, 3:end]
```

```
Out[103]: 3-element Vector{Int64}:  
 18  
 13  
 16
```

Рисунок 4.11: Массивы

```
In [104]: sort(ar,dims=1)
```

```
Out[104]: 10×5 Matrix{Int64}:  
 10  10  10  10  10  
 10  11  15  12  10  
 11  11  16  13  11  
 13  12  17  13  13  
 13  12  18  14  15  
 15  16  18  14  15  
 15  17  18  16  16  
 17  19  19  16  16  
 18  20  19  18  16  
 19  20  20  18  20
```

```
In [105]: sort(ar,dims=2)
```

```
Out[105]: 10×5 Matrix{Int64}:  
 13  15  16  18  19  
 10  10  11  15  18  
 12  15  18  19  19  
 12  13  15  16  16  
 13  14  17  18  20  
 10  10  18  20  20  
 10  10  12  15  18  
 13  13  16  17  20  
 11  16  16  17  19  
 10  11  11  14  16
```

Рисунок 4.12: Массивы

```

In [106]: ar .> 14
Out[106]: 10x5 BitMatrix:
  1 1 1 0 1
  0 0 1 0 1
  1 0 1 1 1
  1 0 1 0 1
  0 1 1 0 1
  0 1 1 1 0
  1 0 1 0 0
  0 1 1 1 0
  1 1 1 1 0
  0 0 0 0 1

In [107]: findall(ar .> 14)
Out[107]: 29-element Vector{CartesianIndex{2}}:
 CartesianIndex(1, 1)
 CartesianIndex(3, 1)
 CartesianIndex(4, 1)
 CartesianIndex(7, 1)
 CartesianIndex(9, 1)
 CartesianIndex(1, 2)
 CartesianIndex(5, 2)
 CartesianIndex(6, 2)
 CartesianIndex(8, 2)
 CartesianIndex(9, 2)
 CartesianIndex(1, 3)
 CartesianIndex(2, 3)
 CartesianIndex(3, 3)
      ⋮
 CartesianIndex(8, 3)
 CartesianIndex(9, 3)
 CartesianIndex(3, 4)
 CartesianIndex(6, 4)
 CartesianIndex(8, 4)
 CartesianIndex(9, 4)
 CartesianIndex(1, 5)
 CartesianIndex(2, 5)
 CartesianIndex(3, 5)
 CartesianIndex(4, 5)
 CartesianIndex(5, 5)
 CartesianIndex(10, 5)

```

Рисунок 4.13: Массивы

Далее, необходимо было выполнить *Задания для самостоятельного выполнения*:

1. Даны множества:

- $\mathbb{X} = \{0, 3, 4, 9\}$,
- $\mathbb{X} = \{1, 3, 4, 7\}$,
- $\mathbb{X} = \{0, 1, 2, 4, 7, 8, 9\}$.

Найти

$$P = A \cap B \cup A \cap B \cup A \cap C \cup B \cap C$$

(рис. 4.14).

1. Операции над множествами

```
In [110]: A = Set([0, 3, 4, 9])
          B = Set([1, 3, 4, 7])
          C = Set([0, 1, 2, 4, 7, 8, 9])
          P = union(intersect(A, B), intersect(A, B), intersect(A, C), intersect(B, C))

Out[110]: Set{Int64} with 6 elements:
           0
           4
           7
           9
           3
           1
```

Рисунок 4.14: Объединение множеств

2. Приведите свои примеры с выполнением операций над множествами элементов разных типов (рис. 4.15).

2. Собственные примеры

```
In [113]: S1 = Set(["apple", 42, 3.14])
          S2 = Set([42, "banana", 2.75])

Out[113]: Set{Any} with 3 elements:
           2.75
           "banana"
           42

In [114]: union(S1, S2)

Out[114]: Set{Any} with 5 elements:
           2.75
           3.14
           42
           "banana"
           "apple"

In [115]: intersect(S1, S2)

Out[115]: Set{Any} with 1 element:
           42

In [116]: setdiff(S1, S2)

Out[116]: Set{Any} with 2 elements:
           3.14
           "apple"

In [117]: push!(S1, 125)

Out[117]: Set{Any} with 4 elements:
           3.14
           42
           "apple"
           125

In [118]: pop!(S2)

Out[118]: 2.75
```

Рисунок 4.15: Собственные примеры

3. Создайте разными способами:

- 1) массив $(1, 2, 3, \dots, N-1, N)$, N выберите больше 20 (рис. 4.16);
- 2) массив $(N, N-1, \dots, 2, 1)$, N выберите больше 20 (рис. 4.16);
- 3) массив $(1, 2, 3, \dots, N-1, N, N-1, \dots, 2, 1)$, N выберите больше 20 (рис. 4.16);
- 4) массив с именем `tmp` вида $(4, 6, 3)$ (рис. 4.16);
- 5) массив, в котором первый элемент массива `tmp` повторяется 10 раз (рис. 4.16);
- 6) массив, в котором все элементы массива `tmp` повторяются 10 раз (рис. 4.16);
- 7) массив, в котором первый элемент массива `tmp` встречается 11 раз, второй элемент — 10 раз, третий элемент — 10 раз (рис. 4.16);
- 8) массив, в котором первый элемент массива `tmp` встречается 10 раз подряд, второй элемент — 20 раз подряд, третий элемент — 30 раз подряд (рис. 4.16);
- 9) массив из элементов вида $2^{tmp[i]}$, $i = 1, 2, 3$, где элемент $2^{tmp[3]}$ встречается 4 раза; посчитайте в полученном векторе, сколько раз встречается цифра 6, и выведите это значение на экран (рис. 4.16);
- 10) вектор значений $y = e^x \cos(x)$ в точках $x = 3, 3.1, 3.2, \dots, 6$, найдите среднее значение y (рис. 4.17);
- 11) вектор вида (x^i, y^j) , $x = 0.1, i = 3, 6, 9, \dots, 36, y = 0.2, j = 1, 4, 7, \dots, 34$ (рис. 4.18);
- 12) вектор с элементами $\frac{2^i}{i}$, $i = 1, 2, \dots, M, M = 25$ (рис. 4.19);
- 13) вектор вида $(\text{"fn1"}, \text{"fn2"}, \dots, \text{"fnN"})$, $N = 30$ (рис. 4.19);

14) векторы $x = (x_1, x_2, \dots, x_n)$ $y = (y_1, y_2, \dots, y_n)$ целочисленного типа длины $n = 250$ как случайные выборки из совокупности $0, 1, \dots, 999$; на его основе (рис. 4.20):

- сформируйте вектор $(y_2 - x_1, \dots, y_n - x_{n-1})$ (рис. 4.20);
- сформируйте вектор $(x_1 + 2x_2 - x_3, x_2 + 2x_3 - x_4, \dots, x_{n-2} + 2x_{n-1} - x_n)$ (рис. 4.20);
- сформируйте вектор $(\sin(y_1)/\cos(x_2), \sin(y_2)/\cos(x_3), \dots, \sin(y_{n-1})/\cos(x_n))$ (рис. 4.20);
- вычислите $\sum_{i=1}^{n-1} e^{-x_{i+1}}/(x_i + 10)$ (рис. 4.20);
- выберите элементы вектора $\mathbb{X}$, значения которых больше 600, и выведите на экран; определите индексы этих элементов (рис. 4.21);
- определите значения вектора $\mathbb{X}$, соответствующие значениям вектора $\mathbb{X}$, значения которых больше 600 (под соответствием понимается расположение на аналогичных индексных позициях) (рис. 4.21);
- сформируйте вектор $(|x_1 - x|^{1/2}, |x_2 - x|^{1/2}, \dots, |x_n - x|^{1/2})$, где x обозначает среднее значение вектора $x = (x_1, x_2, \dots, x_n)$ (рис. 4.22);
- определите, сколько элементов вектора y отстоят от максимального значения не более, чем на 200 (рис. 4.22);
- определите, сколько чётных и нечётных элементов вектора x (рис. 4.22);
- определите, сколько элементов вектора x кратны 7 (рис. 4.22);
- отсортируйте элементы вектора x в порядке возрастания элементов вектора y (рис. 4.23);
- выведите элементы вектора x , которые входят в десятку наибольших (top-10) (рис. 4.24);
- сформируйте вектор, содержащий только уникальные (неповторяющиеся) элементы вектора x (рис. 4.24).

4. Создайте массив `squares`, в котором будут храниться квадраты всех целых

чисел от 1 до 100 (рис. 4.24).

- Подключите пакет `Primes` (функции для вычисления простых чисел). Сгенерируйте массив `myprimes`, в котором будут храниться первые 168 простых чисел. Определите 89-е наименьшее простое число. Получите срез массива с 89-го до 99-го элемента включительно, содержащий наименьшие простые числа (рис. 4.25).

- Вычислите следующие выражения:

- $\sum_{i=10}^1 00_{i=10}(i^3 + 4i^2)$ (рис. 4.26);
- $\sum_{i=1}^M (2^i/i + 3^i/i^2)$, $M = 25$ (рис. 4.26);
- $1 + 2/3 + (2/34/5) + (2/34/56/7) + \dots + (2/34/5\dots 38/39)$ (рис. 4.26);

```

3. Работа с массивами
!!! Для удобства вывода информации - буду транспонировать массивы, где это возможно !!!

In [128]: N=24
Out[128]: 24

In [126]: a = transpose([i for i in 1:N])
Out[126]: 1x24 transpose(::Vector{Int64}) with eltype Int64:
           1 2 3 4 5 6 7 8 9 10 11 12 .. 16 17 18 19 20 21 22 23 24

In [127]: b = transpose(collect(N:-1:1))
Out[127]: 1x24 transpose(::Vector{Int64}) with eltype Int64:
           24 23 22 21 20 19 18 17 16 15 .. 11 10 9 8 7 6 5 4 3 2 1

In [129]: transpose(vcat([i for i in 1:N-1], collect(N:-1:1)))
Out[129]: 1x47 transpose(::Vector{Int64}) with eltype Int64:
           1 2 3 4 5 6 7 8 9 10 11 12 .. 11 10 9 8 7 6 5 4 3 2 1

In [130]: tmp = [4,6,3]
Out[130]: 3-element Vector{Int64}:
           4
           6
           3

In [141]: transpose(fill(tmp[1], (10, 1)))
Out[141]: 1x10 transpose(::Matrix{Int64}) with eltype Int64:
           4 4 4 4 4 4 4 4 4 4

In [140]: transpose(repeat(tmp, inner=10))
Out[140]: 1x30 transpose(::Vector{Int64}) with eltype Int64:
           4 4 4 4 4 4 4 4 4 4 6 6 6 6 .. 6 6 3 3 3 3 3 3 3 3 3 3

In [144]: transpose(vcat(fill(tmp[1], (11, 1)), fill(tmp[2], (10, 1)), fill(tmp[3], (10, 1))))
Out[144]: 1x31 transpose(::Matrix{Int64}) with eltype Int64:
           4 4 4 4 4 4 4 4 4 4 6 6 .. 6 6 3 3 3 3 3 3 3 3 3 3

In [145]: transpose(vcat(fill(tmp[1], (10, 1)), fill(tmp[2], (20, 1)), fill(tmp[3], (30, 1))))
Out[145]: 1x60 transpose(::Matrix{Int64}) with eltype Int64:
           4 4 4 4 4 4 4 4 4 4 6 6 6 .. 3 3 3 3 3 3 3 3 3 3 3 3

In [155]: ar = transpose(vcat(fill(2*tmp[1], (1,1)), fill(2*tmp[2], (1,1)), fill(2*tmp[3], (4,1))))
Out[155]: 1x6 transpose(::Matrix{Int64}) with eltype Int64:
           16 64 8 8 8 8

In [159]: count(x -> x == 6, ar)
Out[159]: 0

```

Рисунок 4.16: Работа с массивами

```

In [172]: x = transpose(collect(3:0.1:6))
Out[172]: 1x31 transpose(::Vector{Float64}) with eltype Float64:
          3.0  3.1  3.2  3.3  3.4  3.5  3.6  3.7  -  5.4  5.5  5.6  5.7  5.8  5.9  6.0

In [174]: y = exp.(x) .* cos.(x)
Out[174]: 1x31 Matrix{Float64}:
          -19.8845 -22.1788 -24.4907 -26.7732 - 249.468 292.487 338.564 387.36

In [175]: using Statistics
          mean(y)
Out[175]: 53.11374594642971

```

Рисунок 4.17: Работа с массивами

```

In [177]: i = collect(3:3:36)
          j = collect(1:3:34)
Out[177]: 12-element Vector{Int64}:
           1
           4
           7
          10
          13
          16
          19
          22
          25
          28
          31
          34

In [185]: x = 0.1
          y = 0.2
          xy = reshape(vcat([x^i for i in i], [y^j for j in j]), (12, 2))
Out[185]: 12x2 Matrix{Float64}:
           0.001  0.2
           1.0e-6  0.0016
           1.0e-9  1.28e-5
           1.0e-12  1.024e-7
           1.0e-15  8.192e-10
           1.0e-18  6.5536e-12
           1.0e-21  5.24288e-14
           1.0e-24  4.1943e-16
           1.0e-27  3.35544e-18
           1.0e-30  2.68435e-20
           1.0e-33  2.14748e-22
           1.0e-36  1.71799e-24

```

Рисунок 4.18: Работа с массивами


```

In [195]: M = 25
          [2^i/i for i in 1:M]

Out[195]: 25-element Vector{Float64}:
           2.0
           2.0
          2.6666666666666665
           4.0
           6.4
          10.666666666666666
          18.285714285714285
          32.0
          56.888888888888886
          102.4
          186.1818181818182
          341.3333333333333
          630.1538461538462
          1170.2857142857142
          2184.5333333333333
          4096.0
          7710.117647058823
          14563.555555555555
          27594.105263157893
          52428.8
          99864.38095238095
          190650.18181818182
          364722.0869565217
          699050.6666666666
           1.34217728e6

In [197]: N = 30
          [string("fn", string(i)) for i in 1:N]

Out[197]: 30-element Vector{String}:
           "fn1"
           "fn2"
           "fn3"
           "fn4"
           "fn5"
           "fn6"
           "fn7"
           "fn8"
           "fn9"
           "fn10"
           "fn11"
           "fn12"
           "fn13"
           ⋮
           "fn19"
           "fn20"
           "fn21"
           "fn22"
           "fn23"
           "fn24"
           "fn25"
           "fn26"
           "fn27"
           "fn28"
           "fn29"
           "fn30"

```

Рисунок 4.19: Работа с массивами

```

In [193]: n = 250
          x = rand(0:999, n)
          y = rand(0:999, n)

Out[193]: 250-element Vector{Int64}:
          387
          767
           30
          775
          568
          382
          364
          745
          305
           24
          971
          268
           520
           1
          814
          264
          609
          930
          879
          431
          522
          697
          371
           52
          987
           1

In [207]: transpose([y[i+1]-x[i] for i in 1:n-1])

Out[207]: 1x249 transpose(::Vector{Int64}) with eltype Int64:
          279  -516  416  -382  -613  27  -182  ...  -285  -241  368  -129  577  -752

In [208]: transpose([x[i] + 2*x[i+1] - x[i+2] for i in 1:n-2])

Out[208]: 1x248 transpose(::Vector{Int64}) with eltype Int64:
          1221  394  1184  2363  662  1389  1787  ...  1548  2672  779  -37  248  1385

In [210]: transpose([sin(y[i])/cos(x[i+1]) for i in 1:n-1])

Out[210]: 1x249 transpose(::Vector{Float64}) with eltype Float64:
          -0.686846  0.667388  1.0127  -1.18251  ...  -12.9952  1.77718  -0.515349

In [212]: transpose([exp(-x[i+1])/(x[i]+10) for i in 1:n-1])

Out[212]: 1x249 transpose(::Vector{Float64}) with eltype Float64:
          1.58655e-248  2.20397e-159  0.0  0.0  ...  4.55228e-181  0.0  3.21444e-234

```

Рисунок 4.20: Работа с массивами

```
In [217]: [[i, y[i]] for i in 1:n if y[i] > 600]
```

```
Out[217]: 84-element Vector{Vector{Int64}}:  
 [2, 767]  
 [4, 775]  
 [8, 745]  
 [11, 971]  
 [18, 816]  
 [19, 810]  
 [23, 999]  
 [24, 808]  
 [25, 808]  
 [33, 694]  
 [34, 894]  
 [35, 850]  
 [45, 827]  
 ⋮  
 [223, 899]  
 [224, 900]  
 [225, 930]  
 [227, 814]  
 [231, 705]  
 [236, 748]  
 [239, 814]  
 [241, 609]  
 [242, 930]  
 [243, 879]  
 [246, 697]  
 [249, 987]
```

```
In [220]: [x[i] for i in 1:n if y[i] > 600]
```

```
Out[220]: 84-element Vector{Int64}:  
 546  
 870  
 802  
 779  
 208  
 521  
 539  
 344  
 403  
 756  
 402  
 249  
 86  
 ⋮  
 503  
 692  
 538  
 266  
 171  
 796  
 887  
 11  
 512  
 864  
 11  
 753
```

Рисунок 4.21: Работа с массивами

```

In [221]: x_mean = mean(x)
Out[221]: 497.776

In [222]: [abs(x[i] - x_mean)^0.5 for i in 1:n]
Out[222]: 250-element Vector{Float64}:
 3.12665955933805
 6.944350221583009
11.780322576228548
19.29310757757806
20.426061783907343
12.679747631557973
20.71772188248505
17.442018231844617
15.691526375722662
19.11083462332297
16.76973464310035
19.843789960589685
13.106334346414332
 ⋮
19.72876073148032
 4.333128200272871
22.063000702533643
 3.771471861223412
19.136979908021015
17.58476613435618
20.981515674516938
22.063000702533643
17.798202156397707
 9.36888467214748
15.975731595141426
 5.764026370515665

In [224]: y_max = maximum(y)
Out[224]: 999

In [225]: count(abs(y_i - y_max) <= 200 for y_i in y)
Out[225]: 45

In [227]: count(x_i -> x_i % 2 == 0, x)
Out[227]: 132

In [228]: count(x_i -> x_i % 2 == 1, x)
Out[228]: 118

In [230]: count(x_i -> x_i % 7 == 0, x)
Out[230]: 37

```

Рисунок 4.22: Работа с массивами

```
In [231]: sort_ind = sortperm(y)
x[sort_ind]
```

```
Out[231]: 250-element Vector{Int64}:
 717
 38
 531
 983
 661
 795
 734
 863
 359
 269
 694
 828
 385
 ⋮
 901
 223
 277
 550
 344
 779
 604
 276
 753
 833
 302
 539
```

```
In [232]: y[sort_ind]
```

```
Out[232]: 250-element Vector{Int64}:
 0
 1
 1
 3
 6
 17
 18
 24
 30
 30
 33
 34
 35
 ⋮
 949
 955
 957
 965
 968
 971
 978
 978
 987
 993
 996
 999
```

Рисунок 4.23: Работа с массивами

```
In [233]: sort(x, rev=true)[1:10]
```

```
Out[233]: 10-element Vector{Int64}:  
 988  
 984  
 983  
 974  
 968  
 963  
 950  
 947  
 942  
 938
```

```
In [234]: unique(x)
```

```
Out[234]: 226-element Vector{Int64}:  
 488  
 546  
 359  
 870  
 915  
 337  
 927  
 802  
 744  
 863  
 779  
 104  
 326  
  ⋮  
 796  
 115  
 579  
 479  
  11  
 512  
 864  
 807  
 181  
 410  
 753  
 531
```

```
In [236]: squares = [i^2 for i in 1:100]
```

```
Out[236]: 100-element Vector{Int64}:  
  1  
  4  
  9  
 16  
 25  
 36  
 49  
 64  
 81  
100  
121  
144  
169  
  ⋮
```

Рисунок 4.24: Работа с массивами

```
In [240]: using Primes

In [244]: myprimes = primes(1000)
Out[244]: 168-element Vector{Int64}:
 2
 3
 5
 7
11
13
17
19
23
29
31
37
41
 ⋮
919
929
937
941
947
953
967
971
977
983
991
997

In [245]: myprimes[89]
Out[245]: 461

In [246]: myprimes[89:99]
Out[246]: 11-element Vector{Int64}:
 461
 463
 467
 479
 487
 491
 499
 503
 509
 521
 523

In [239]: import Pkg; Pkg.add("Primes")
```

Рисунок 4.25: Работа с массивами

```

In [247]: sum(i^3 + 4*i^2 for i in 10:100)
Out[247]: 26852735

In [248]: M = 25
           sum((2^i)/i + (3^i)/(i^2) for i in 1:M)
Out[248]: 2.1291704368143802e9

In [249]: aa = ones(19)
Out[249]: 19-element Vector{Float64}:
           1.0
           1.0
           1.0
           1.0
           1.0
           1.0
           1.0
           1.0
           1.0
           1.0
           1.0
           1.0
           1.0
           1.0
           1.0
           1.0
           1.0
           1.0

In [250]: aa[1] = 2/3
Out[250]: 0.6666666666666666

In [254]: for i in 2:19
           aa[i] = aa[i-1] * (2*i)/(2*i + 1)
           end

In [256]: 1 + sum(aa)
Out[256]: 6.97634613789762

```

Рисунок 4.26: Работа с массивами

5 Выводы

В ходе работы были изучены основы работы с разными типами структур в я.п. Julia.

Список литературы